

Propuesta de Proyecto

Desarrollo de Videojuego Tower Defense

Algoritmos y Programación II

Docente: Jonathan Hernández

Programa: Ingeniería de Sistemas

Semestre: 2026A

8 de abril de 2026

1. Introducción

El presente documento describe la propuesta del proyecto final para el curso de Algoritmos y Programación II, centrado en el desarrollo de un videojuego del género Tower Defense utilizando los principios de Programación Orientada a Objetos (POO) en Java.

El proyecto tiene como objetivo principal la aplicación práctica de conceptos fundamentales de POO, patrones de diseño, persistencia de datos, testing y gestión de proyectos a través del desarrollo de un sistema de entretenimiento interactivo.

2. Objetivos

2.1. Objetivo General

Desarrollar un videojuego Tower Defense funcional que demuestre la comprensión y aplicación de los conceptos de Programación Orientada a Objetos, patrones de diseño y buenas prácticas de desarrollo de software.

2.2. Objetivos Específicos

1. Implementar una jerarquía de clases que represente torres, enemigos y elementos del juego
2. Aplicar principios de POO: herencia, polimorfismo, encapsulación y abstracción
3. Desarrollar un sistema de persistencia utilizando base de datos

4. Implementar patrones de diseño apropiados (MVC, Factory, Strategy, Observer)
5. Gestionar el proyecto utilizando Maven
6. Desarrollar pruebas unitarias con JUnit
7. Crear una interfaz gráfica funcional e intuitiva
8. Trabajar colaborativamente siguiendo metodologías ágiles

3. Descripción del Proyecto

3.1. Concepto del Juego

Título: A definir por cada equipo de trabajo

Género: Tower Defense con elementos de estrategia

Temática: Libre elección del equipo. Algunas opciones sugeridas incluyen:

- Fantasy medieval (magos, dragones, castillos)
- Ciencia ficción (robots, aliens, bases espaciales)
- Histórica (civilizaciones antiguas, batallas épicas)
- Post-apocalíptica (zombies, supervivientes, refugios)
- Steampunk (máquinas de vapor, inventos victorianos)
- Cyberpunk (hackers, corporaciones, ciudades futuristas)

El equipo debe desarrollar una temática coherente que incluya:

- Contexto narrativo del conflicto
- Justificación para las torres defensivas
- Motivación de los enemigos atacantes
- Ambientación visual y sonora apropiada
- Elementos únicos que distingan su propuesta

3.2. Narrativa del Videojuego

Cada equipo debe desarrollar una narrativa original que se adapte a la temática elegida. La historia debe incluir los siguientes elementos estructurales:

3.2.1. Elementos Narrativos Requeridos

- **Contexto inicial:** Situación de paz o equilibrio que será alterada
- **Conflicto principal:** Amenaza externa que justifica la necesidad de defensa
- **Protagonista:** Rol del jugador como comandante, estratega o defensor
- **Antagonista:** Fuerza invasora con motivaciones claras
- **Objetivo principal:** Elemento a proteger (base, ciudad, recurso, etc.)

3.2.2. Estructura Narrativa Sugerida

1. **Planteamiento:** Presentación del mundo y la situación inicial
2. **Incidente desencadenante:** Evento que inicia el conflicto
3. **Desarrollo:** Escalada progresiva de la amenaza (oleadas)
4. **Climax:** Enfrentamiento con enemigos finales o jefes
5. **Resolución:** Victoria del jugador y restauración del orden

3.2.3. Integración con Mecánicas de Juego

La narrativa debe justificar coherentemente:

- Los diferentes tipos de torres defensivas disponibles
- Las características y comportamientos de los enemigos
- El sistema de recursos y mejoras
- La progresión de dificultad a través de las oleadas
- Los elementos visuales y estéticos del juego

Nota: La narrativa será evaluada en la primera entrega parcial y debe estar completamente documentada antes del desarrollo técnico.

3.3. Mecánicas de Juego

Cada equipo debe diseñar e implementar las mecánicas específicas de su Tower Defense, manteniendo los elementos core del género. Las siguientes son las especificaciones mínimas requeridas:

3.3.1. Torres Defensivas (Mínimo 4 tipos)

Cada equipo debe implementar al menos cuatro tipos de torres con características diferenciadas:

- **Torre Tipo A:** Especialización y características únicas
- **Torre Tipo B:** Especialización y características únicas
- **Torre Tipo C:** Especialización y características únicas
- **Torre Tipo D:** Especialización y características únicas

Atributos requeridos por torre:

- Daño base y tipo de daño
- Alcance de ataque
- Velocidad de disparo
- Costo de construcción y mejora
- Efectos especiales (opcional)

3.3.2. Tipos de Enemigos (Mínimo 5 tipos)

El sistema debe incluir al menos cinco tipos de enemigos con comportamientos distintos:

- **Enemigo Básico:** Características estándar
- **Enemigo Rápido:** Alta velocidad, baja resistencia
- **Enemigo Resistente:** Baja velocidad, alta resistencia
- **Enemigo Especial:** Habilidad única (volar, regenerar, etc.)
- **Enemigo Jefe:** Múltiples mecánicas y alta dificultad

Atributos requeridos por enemigo:

- Puntos de vida
- Velocidad de movimiento
- Resistencias y vulnerabilidades
- Recompensa por eliminación
- Habilidades especiales

3.3.3. Mecánicas Core del Sistema

- **Sistema de Recursos:** Moneda/energía para construir y mejorar torres
- **Sistema de Oleadas:** Progresión de dificultad con patrones de enemigos
- **Sistema de Vida/Salud:** Penalización por enemigos que llegan al objetivo
- **Sistema de Puntuación:** Evaluación del desempeño del jugador
- **Condiciones de Victoria/Derrota:** Criterios claros de fin de partida

3.3.4. Mecánicas Opcionales (Puntos Extra)

- Sistema de mejoras/upgrades para torres
- Habilidades especiales activables por el jugador
- Múltiples mapas o rutas
- Sistema de logros o desbloques
- Efectos de sonido y música

Nota: Las mecánicas específicas deben estar documentadas en el diagrama de clases y ser coherentes con la narrativa elegida.

4. Arquitectura y Diseño

4.1. Diagrama de Clases Principal

El sistema estará basado en las siguientes clases principales:

- **Entidad** (clase abstracta)
 - **Torre** (clase abstracta)
 - TorreArquero
 - TorreMagia
 - TorreCañon
 - TorreHielo
 - **Enemigo** (clase abstracta)
 - SoldadoBasico
 - Berserker
 - TanqueBlindado
 - EnemigoVolador
 - Jefe
- **Juego** (controlador principal)
- **Mapa** (gestión del tablero)
- **Oleada** (gestión de enemigos)
- **Jugador** (recursos y puntuación)
- **InterfazGrafica** (vista del juego)

4.2. Patrones de Diseño a Implementar

- **MVC (Model-View-Controller):** Separación de lógica, vista y control
- **Factory Method:** Creación de torres y enemigos
- **Strategy:** Diferentes comportamientos de IA para enemigos
- **Observer:** Notificaciones de eventos del juego
- **Singleton:** Gestión del estado global del juego

5. Cronograma de Desarrollo

Fecha	Entregable	Responsables
08 abril 2026	Entrega Parcial 1: ▪ Diagrama de clases inicial ▪ Documento de narrativa ▪ Definición de arquitectura	Todo el equipo
29 abril 2026	ENTREGA MAYOR 1: ▪ Clases principales implementadas ▪ Primeros sprites y recursos gráficos ▪ Sistema básico de persistencia (BD) ▪ Presentación con diapositivas	Todo el equipo
13 mayo 2026	Entrega Parcial 2: ▪ Sistema de torres funcional ▪ Mecánica básica de enemigos ▪ Configuración de Maven ▪ Primeras pruebas unitarias	Todo el equipo
27 mayo 2026	ENTREGA FINAL: ▪ Prototipo funcional completo ▪ Implementación de patrones MVC ▪ Suite completa de pruebas JUnit ▪ Documentación técnica ▪ Presentación final con demo	Todo el equipo

6. Criterios de Evaluación

6.1. Distribución de Calificaciones

Componente	Porcentaje	Descripción
Entrega Parcial 1 (8 abril)	10 %	Diseño y planificación inicial
Entrega Mayor 1 (29 abril)	30 %	Implementación básica y BD
Entrega Parcial 2 (13 mayo)	15 %	Funcionalidades core y testing
Entrega Final (27 mayo)	35 %	Producto completo y patrones
Presentaciones	10 %	Comunicación y demostración

6.2. Rúbrica de Evaluación

6.2.1. Implementación de POO (25 puntos)

- **Excelente (23-25):** Uso correcto y avanzado de herencia, polimorfismo, encapsulación y abstracción

- **Bueno (18-22):** Implementación correcta de la mayoría de conceptos POO
- **Regular (13-17):** Implementación básica con algunos errores conceptuales
- **Deficiente (0-12):** Pobre aplicación de principios POO

6.2.2. Patrones de Diseño (20 puntos)

- **Excelente (18-20):** Implementación correcta de MVC, Factory, Strategy y Observer
- **Bueno (14-17):** Implementación de al menos 3 patrones correctamente
- **Regular (10-13):** Implementación básica de 2 patrones
- **Deficiente (0-9):** Implementación incorrecta o ausente de patrones

6.2.3. Funcionalidad del Juego (20 puntos)

- **Excelente (18-20):** Juego completamente funcional con todas las mecánicas
- **Bueno (14-17):** Funcionalidades principales implementadas
- **Regular (10-13):** Funcionalidad básica con limitaciones
- **Deficiente (0-9):** Juego no funcional o con errores críticos

6.2.4. Persistencia y Base de Datos (15 puntos)

- **Excelente (14-15):** Sistema completo de BD con operaciones CRUD
- **Bueno (11-13):** Implementación correcta de persistencia básica
- **Regular (8-10):** Funcionalidad limitada de BD
- **Deficiente (0-7):** Sistema de BD no funcional

6.2.5. Testing y Calidad de Código (10 puntos)

- **Excelente (9-10):** Cobertura completa con JUnit y código limpio
- **Bueno (7-8):** Pruebas principales implementadas
- **Regular (5-6):** Pruebas básicas con cobertura limitada
- **Deficiente (0-4):** Ausencia de testing o código de baja calidad

6.2.6. Gestión de Proyecto (10 puntos)

- **Excelente (9-10):** Uso correcto de Maven, documentación completa
- **Bueno (7-8):** Gestión adecuada con documentación básica
- **Regular (5-6):** Gestión básica del proyecto
- **Deficiente (0-4):** Pobre gestión y documentación

7. Checklist de Autoevaluación

7.1. Para Entrega Parcial 1 (16 Oct)

- ☐ Diagrama de clases UML completo y detallado
- ☐ Documento de narrativa del videojuego finalizado
- ☐ Definición clara de la arquitectura del sistema
- ☐ Distribución de tareas entre miembros del equipo
- ☐ Configuración inicial del repositorio de código

7.2. Para Entrega Mayor 1 (30 Oct)

- ☐ Clases principales implementadas (Torre, Enemigo, Entidad)
- ☐ Al menos 3 tipos de torres funcionales
- ☐ Al menos 3 tipos de enemigos implementados
- ☐ Sprites básicos integrados en el proyecto
- ☐ Sistema de base de datos configurado y funcional
- ☐ Operaciones CRUD básicas implementadas
- ☐ Presentación preparada con demo básico
- ☐ Código versionado y documentado

7.3. Para Entrega Parcial 2 (13 Nov)

- ☐ Sistema de torres completamente funcional
- ☐ Mecánica de enemigos con movimiento y ataque
- ☐ Proyecto configurado correctamente con Maven
- ☐ Dependencias gestionadas apropiadamente
- ☐ Suite inicial de pruebas unitarias con JUnit
- ☐ Cobertura de testing de al menos 60 %
- ☐ Sistema básico de oleadas implementado
- ☐ Interfaz gráfica funcional

7.4. Para Entrega Final (27 Nov)

- ☐ Juego completamente funcional y jugable
- ☐ Patrón MVC correctamente implementado
- ☐ Todos los patrones de diseño requeridos funcionando
- ☐ Suite completa de pruebas JUnit (cobertura ¿80 %)
- ☐ Sistema de persistencia completo y robusto
- ☐ Documentación técnica completa
- ☐ Manual de usuario incluido
- ☐ Presentación final con demo completo
- ☐ Código limpio y bien comentado
- ☐ Sistema de build automático funcionando

8. Tecnologías y Herramientas

8.1. Herramientas de Desarrollo

- **Lenguaje:** Java 11 o superior
- **IDE:** IntelliJ IDEA, Eclipse o NetBeans
- **Gestión de Proyecto:** Apache Maven
- **Testing:** JUnit 5
- **Base de Datos:** SQLite o MySQL
- **Control de Versiones:** Git
- **Interfaz Gráfica:** Java Swing o JavaFX

8.2. Bibliotecas Recomendadas

- JDBC para conectividad con base de datos
- Mockito para testing avanzado
- Jackson para serialización JSON (opcional)
- Log4j para logging del sistema

9. Recursos y Referencias

9.1. Documentación Técnica

- Oracle Java Documentation
- Maven Official Guide
- JUnit 5 User Guide
- Design Patterns: Elements of Reusable Object-Oriented Software

9.2. Recursos Gráficos

- OpenGameArt.org para sprites libres
- Kenney.nl para assets de juegos
- Freesound.org para efectos de sonido (opcional)